



GUÍA PRÁCTICA N° 11 - Generación de Código Intermedio con C++, Flex y Bison

Curso: Compiladores y teoría de lenguajes

Unidad: Generación de Código intermedio

Duración: 02 horas (laboratorio - Practicas)

Lenguajes utilizados

- C++
- Flex (Analizador Léxico)
- Bison/Yacc (Analizador Sintáctico)

II. LOGRO DE APRENDIZAJE

Al finalizar la práctica, el estudiante implementará un generador de código intermedio utilizando Flex y Bison, capaz de transformar expresiones aritméticas y estructuras de control en código de tres direcciones (Three Address Code - TAC).

III. FUNDAMENTO TEÓRICO

¿Qué es el Código Intermedio?

Es una representación del programa fuente entre el análisis semántico y la generación de código objeto.

Su finalidad es:

- Independizar el compilador del hardware.
- Facilitar las optimizaciones.
- Simplificar la generación del código máquina.

Representaciones del Código Intermedio

Las más utilizadas son:

- Árbol de Sintaxis Abstracta (AST)
- Código Postfijo
- Código de Tres Direcciones (TAC)
- Cuádruplos
- Triplos
- Código SSA

En esta práctica se utilizará:

Código de Tres Direcciones

Ejemplo

$a = b + c * d$

↓

$t1 = c * d$

$t2 = b + t1$

$a = t2$

Variables Temporales

Durante la generación se crean variables temporales.

Ejemplo

t1

t2

t3

...

Ventajas

- Fácil optimización
- Fácil traducción a ensamblador
- Independiente del procesador
- Reduce la complejidad del compilador

IV. IMPLEMENTACIÓN EN C++

Programa para generar código de tres direcciones

```
#include <iostream>
#include <string>
using namespace std;
int temporal = 1;
string nuevoTemporal()
{
    return "t" + to_string(temporal++);
}
int main()
{
    string a,b,c,d;
    cout<<"Ingrese variable destino: ";
    cin>>a;
    cout<<"Ingrese primer operando: ";
    cin>>b;
    cout<<"Ingrese segundo operando: ";
    cin>>c;
    cout<<"Ingrese tercer operando: ";
    cin>>d;
    string t1=nuevoTemporal();
    string t2=nuevoTemporal();

    cout<<"\nCodigo Intermedio\n\n";
```



```
cout<<t1<<" = "<<c<<" * "<<d<<endl;
cout<<t2<<" = "<<b<<" + "<<t1<<endl;
cout<<a<<" = "<<t2<<endl;
return 0;
}
```

Salida

Ingrese variable destino: x
 Ingrese primer operando: a
 Ingrese segundo operando: b
 Ingrese tercer operando: c
 Código Intermedio

```
t1 = b * c
t2 = a + t1
x = t2
```

V. IMPLEMENTACIÓN CON FLEX

Archivo **scanner.l**

```
%{
#include "parser.tab.h"
#include <stdlib.h>
%}
%%
[0-9]+      { yyval.num = atoi(yytext); return NUM; }
[a-zA-Z][a-zA-Z0-9]* { return ID; }
"+" return '+';
"-" return '-';
"*" return '*';
"/" return '/';
"=" return '=';
";" return ';';
[ \t\n] ;
. ;
%%
```

VI. IMPLEMENTACIÓN CON BISON/YACC

Archivo **parser.y**

```
%{
#include<iostream>
#include<string>

using namespace std;
```



```
int temp=1;
string nuevoTemp()
{
    return "t"+to_string(temp++);
}
void yyerror(const char *s);
int yylex();
%}
%union{
    int num;
    char *cad;
}
%token NUM
%token ID
%%
programa:
programa sentencia
|
;

sentencia:

ID '=' expresion ';'
;

expresion:
expresion '+' termino
{
    string t=nuevoTemp();
    cout<<t<<" = suma"<<endl;
}
|
termino
;
termino:
termino '*' factor
{
    string t=nuevoTemp();

    cout<<t<<" = multiplicacion"<<endl;
}
|
```



```
factor
;
factor:
NUM
|
ID
;
%%
void yyerror(const char *s)
{
    cout<<"Error"<<endl;
}
int main()
{
    yyparse();
}
```

VII. GENERACIÓN DE CÓDIGO INTERMEDIO

Entrada

$x = a + b * c;$

Salida esperada

$t1 = b * c$

$t2 = a + t1$

$x = t2$

VIII. CÓDIGO DE TRES DIRECCIONES PARA OPERADORES

Suma

$t1 = a + b$

Resta

$t2 = a - b$

Multiplicación

$t3 = a * b$

División

$t4 = a / b$

Asignación

$a = t4$

IX. GENERACIÓN PARA EXPRESIONES COMPLEJAS



Expresión

$x = (a+b) * (c-d)$

Código intermedio

$t1 = a + b$

$t2 = c - d$

$t3 = t1 * t2$

$x = t3$

X. GENERACIÓN PARA CONDICIONALES

Código fuente

if(a>b)

 x=a;

else

 x=b;

Código intermedio

if a>b goto L1

goto L2

L1:

x=a

goto L3

L2:

x=b

L3:

XI. GENERACIÓN PARA CICLO WHILE

Código

while(i<10)

{

 i=i+1;

}

Código intermedio

L1:

ifFalse i<10 goto L2

t1=i+1

i=t1

goto L1

L2:

XII. GENERACIÓN PARA CICLO FOR

Código

for(i=0;i<5;i++)



Código intermedio

i=0

L1:

if False i<5 goto L2

...

i=i+1

goto L1

L2:

XIII. EJERCICIOS PROPUESTOS

Ejercicio 1

Generar el código intermedio de:

$a = b + c * d - e$

Ejercicio 2

Generar el TAC para

$x = (a + b + c) * d$

Ejercicio 3

Construir el código intermedio para

if (x < y)

z = x + y;

Ejercicio 4

Generar el TAC para

while (i < 20)

i = i + 2;

Ejercicio 5

Implementar en Bison una regla que permita evaluar expresiones con:

+

-

*

/

()

y produzca automáticamente variables temporales.

Ejercicio 6

Desarrollar un compilador simplificado que:

- Analice el código fuente con Flex.
- Construya el árbol sintáctico con Bison.
- Genere código de tres direcciones.



- Muestre las variables temporales creadas.
- Genere etiquetas (L1, L2, etc.) para estructuras de control.

XIV. RETO DE LABORATORIO

Desarrollar un compilador básico que acepte el siguiente programa:

inicio

a=5;

=8;

c=a+b*2;

if(c>10)

d=c*4;

else

d=0;

while(d>0)

d=d-1;

fin

El sistema deberá:

1. Analizar los tokens con **Flex**.
2. Validar la sintaxis mediante **Bison/Yacc**.
3. Construir una representación intermedia.
4. Generar **Código de Tres Direcciones (TAC)**.
5. Mostrar las variables temporales y etiquetas generadas.
6. Preparar el código para una futura fase de optimización o generación de código objeto.

XV. Resultados Esperados

Al finalizar esta guía, el estudiante será capaz de:

- Comprender el propósito del código intermedio en un compilador.
- Implementar un generador de **Código de Tres Direcciones (TAC)** en C++.
- Integrar **Flex** y **Bison/Yacc** para producir código intermedio a partir de expresiones y estructuras de control.
- Generar variables temporales y etiquetas para representar operaciones y flujo de control.
- Diseñar una base sólida para incorporar técnicas de optimización y generación de código objeto en etapas posteriores del desarrollo de un compilador.